

Linear algebra using Python NumPy library

Abdesselam Filali

infos@filali.net

https://github.com/Abdou-fi/python_libraries

Mai 02, 2026

This is a Jupyter notebook converted to L^AT_EX using the nbconvert tool. It introduces linear algebra using the **NumPy** library. For any question or remark please email me or send a request through github.

We start by importing the namespace of the numpy library under the alias np, and by defining an invertible matrix M.

```
[1]: import numpy as np

# for better display of results, precision is set to 4 digits,
# and scientific notation is suppressed.
np.set_printoptions(precision = 4, suppress=True)

# we define the numpy matrix M
M = np.array( [[1, 2, 3],
               [2, 3, 4],
               [3, 4, 6]] )
```

The classical operations related to the resolution of linear systems are available in the linalg library included in numpy.

```
[2]: print(np.linalg.det(M)) # determinant
```

-1.0

```
[3]: M_inv = np.linalg.inv(M) # inverse
print(M_inv)
```

```
[[ -2.   0.   1.]
 [ -0.   3.  -2.]
 [  1.  -2.   1.]]
```

We can check that minv is indeed the (numerical) inverse of M: $M^{-1}M = I$, where I is the identity matrix.

```
[4]: print(np.dot(M_inv, M))
```

```
[[ 1. -0. -0.]
 [ 0.  1.  0.]
```

```
[ 0.  0.  1.]
```

To solve the linear system $MX = B$, we use the function `linalg.solve()`. Note that the matrix must be square and invertible

This function is more efficient than computing the inverse of M and then multiplying it by B , because it uses a more efficient algorithm to solve the system directly.

$MX = B$ is equivalent to the algebraic linear system:

$$\begin{cases} x_1 + 2x_2 + 3x_3 = 2 \\ 2x_1 + 3x_2 + 4x_3 = 2 \\ 3x_1 + 4x_2 + 6x_3 = 3 \end{cases}$$

or the matrix form:

$$\begin{pmatrix} 1 & 2 & 3 \\ 2 & 3 & 4 \\ 3 & 4 & 6 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \\ 3 \end{pmatrix}$$

We can solve this system using numpy:

```
[5]: B = np.array([2, 2, 3])
      X = np.linalg.solve(M, B)
      print(X)
```

```
[-1. -0.  1.]
```

```
[6]: X=[-1, 0, 1] # change from -0 to 0 to avoid Python approximation errors
      print(np.dot(M, X) == B) # we check that the solution is correct
```

```
[ True  True  True]
```

translate to english: On peut aussi calculer les valeurs propres d'une matrice, et les vecteurs propres associés.

$$\det(M\lambda I) = 0$$

```
[7]: eig_val, eig_vec = np.linalg.eig(M) # Eigenvalues & eigen vectors
      print( "Eigenvalues of M :", eig_val)
      print( "Corresponding eigenvectors :")
      for i in range(len(eig_val)):
          print(i, ":", eig_val[i], eig_vec[:,i])
```

```
Eigenvalues of M : [10.1867 -0.4203  0.2336]
Corresponding eigenvectors :
0 : 10.186697673813628 [0.3653 0.5283 0.7665]
1 : -0.42027581011041903 [ 0.9269 -0.1307 -0.3517]
2 : 0.23357813629678265 [ 0.0856 -0.8389  0.5374]
```

We check $MX = \lambda X$

```
[8]: for i in range(len(eig_val)):
      print( np.dot(M, eig_vec[:,i]) - eig_val[i] * eig_vec[:,i])
```

```
[0. 0. 0.]
[-0. -0. -0.]
[-0. -0. -0.]
```

We check $M = UDU^{-1}$, D diagonal

```
[9]: U = eig_vec
      D = np.diag(eig_val)
      Um1 = np.linalg.inv(U)
      print(M, '\n')
      print(np.dot(U, np.dot(D, Um1)))
```

```
[[1 2 3]
 [2 3 4]
 [3 4 6]]
```

```
[[1. 2. 3.]
 [2. 3. 4.]
 [3. 4. 6.]]
```